MATH 8090: Review and Overview of the Course

Whitney Huang, Clemson University

8/20/2026

Contents

Review of Statistical Inference

Inference for a Mean: One-Sample t-Interval

Suppose we observe an independent and identically distributed (i.i.d.) sample

$$X_1, \ldots, X_n$$

from a population with unknown mean μ and variance σ^2. Our goal is to estimate the population mean μ and quantify the uncertainty in that estimate.

We begin by simulating a sample of size $n = 100$ from a standard normal distribution. The function `rnorm(n)` generates n independent observations from a normal distribution with mean 0 and standard deviation 1.

```
# Sample size
n <- 100

# Set a random seed so that the same simulated data are generated
# every time this code is run
set.seed(123)

# Generate 100 independent N(0,1) observations
y <- rnorm(n)

# Display the first few observations
head(y)
```

```
## [1] -0.56047565 -0.23017749  1.55870831  0.07050839  0.12928774  1.71506499
```

```
# Compute the sample mean and sample standard deviation
mean(y)
```

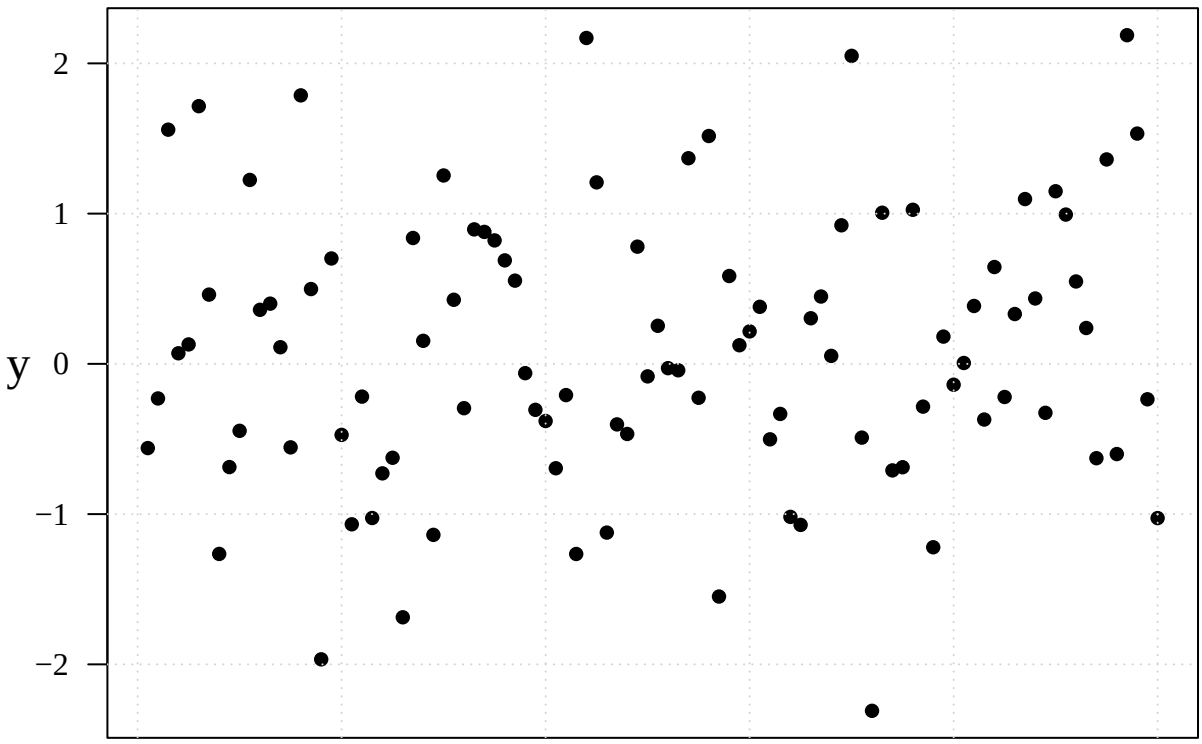
```
## [1] 0.09040591
```

```
sd(y)
```

```
## [1] 0.9128159
```

The following plot displays the 100 observations in the order in which they were generated.

```
par(mar = c(3, 3.5, 0.5, 0.6), mgp = c(2, 1, 0), las = 1, family = "serif")
# Set margins, axis positions, label orientation, and font
plot(y, pch = 16, xlab = "Observation", ylab = "", xaxt = "n", cex.lab = 1.5)
# Plot y using solid points; suppress x-axis tick labels
mtext(expression(y), 2, line = 2, cex = 1.5)
# Add y-axis label
grid() # Add reference grid
```



Observation

We first visualize the simulated observations. The `plot()` function creates a basic scatterplot in `R`. Since y is supplied without an explicit x-variable, `R` automatically uses the observation numbers $1, \cdots, n$ on the horizontal axis. The `par()` function controls the appearance of the plot, while `mtext()` is used here to add a mathematical y label to the vertical axis.

The sample mean $\bar{x}_n$ provides a point estimate of μ, but it does not reflect the uncertainty associated with sampling variability. To quantify this uncertainty, we construct a confidence interval.

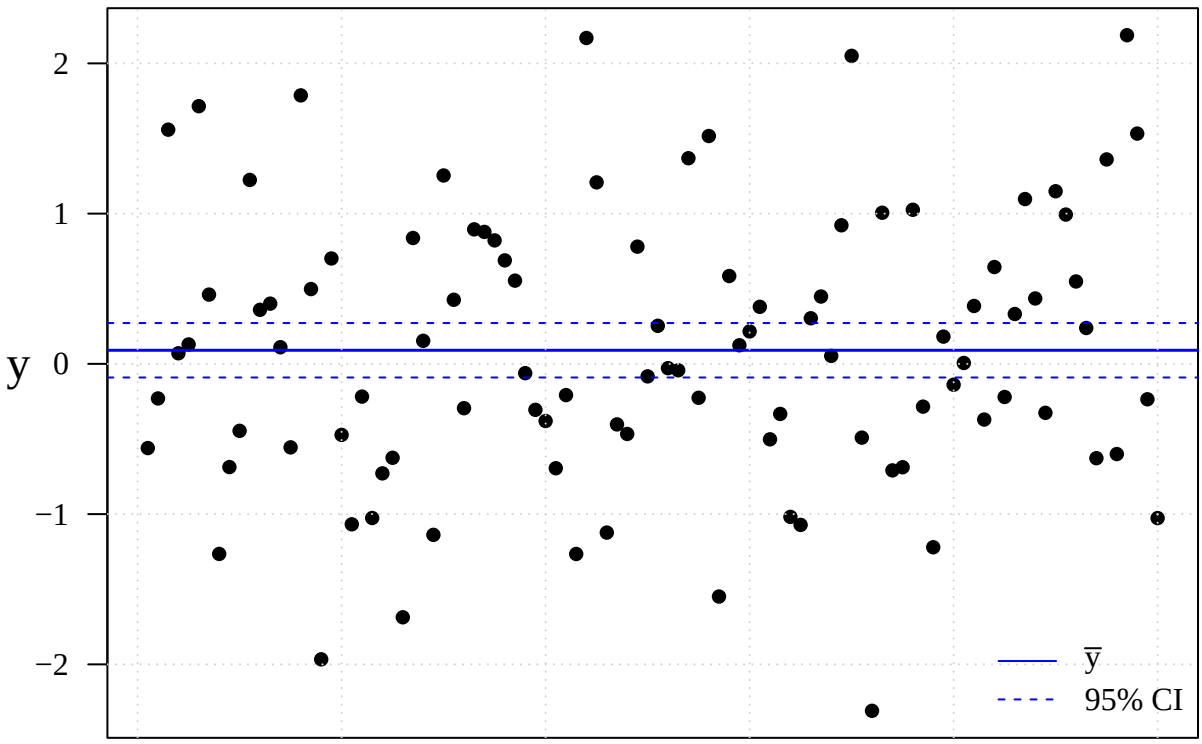
A $(1-\alpha)100\%$ confidence interval for the population mean μ is

$$\bar{x}_n \pm t_{n-1,\,1-\alpha/2}\frac{s}{\sqrt{n}},$$

where $\bar{x}_n$ is the sample mean, s is the sample standard deviation, and $t_{n-1,\,1-\alpha/2}$ is the critical value from a t distribution with $n-1$ degrees of freedom.

For a 95% confidence interval, $\alpha = 0.05$ and the critical value is obtained in R using `qt(0.975, n - 1)`.

```
se <- sd(y) / sqrt(n) # Standard error of the sample mean
tcrit <- qt(0.975, n - 1) # 97.5th percentile of t distribution for a 95% CI

par(mar = c(3, 3.5, 0.5, 0.6), mgp = c(2, 1, 0), las = 1, family = "serif") # Set plot appearance
plot(y, pch = 16, ylab = "", xaxt = "n", cex.lab = 1.5) # Plot observations
mtext(expression(y), 2, line = 2, cex = 1.5) # Add y-axis label
abline(h = mean(y), col = "blue", lwd = 1.5) # Sample mean
abline(h = mean(y) + c(-1, 1) * tcrit * se, col = "blue", lty = 2) # 95% CI limits
grid() # Add reference grid
legend("bottomright", legend = c(expression(bar(y)), "95% CI"), col = "blue", lty = c(1, 2), bty = "n")
```



Index

Inference for Group Means (ANOVA)

We now extend inference for a single population mean to the comparison of **several population means**. Suppose we observe data from k groups,

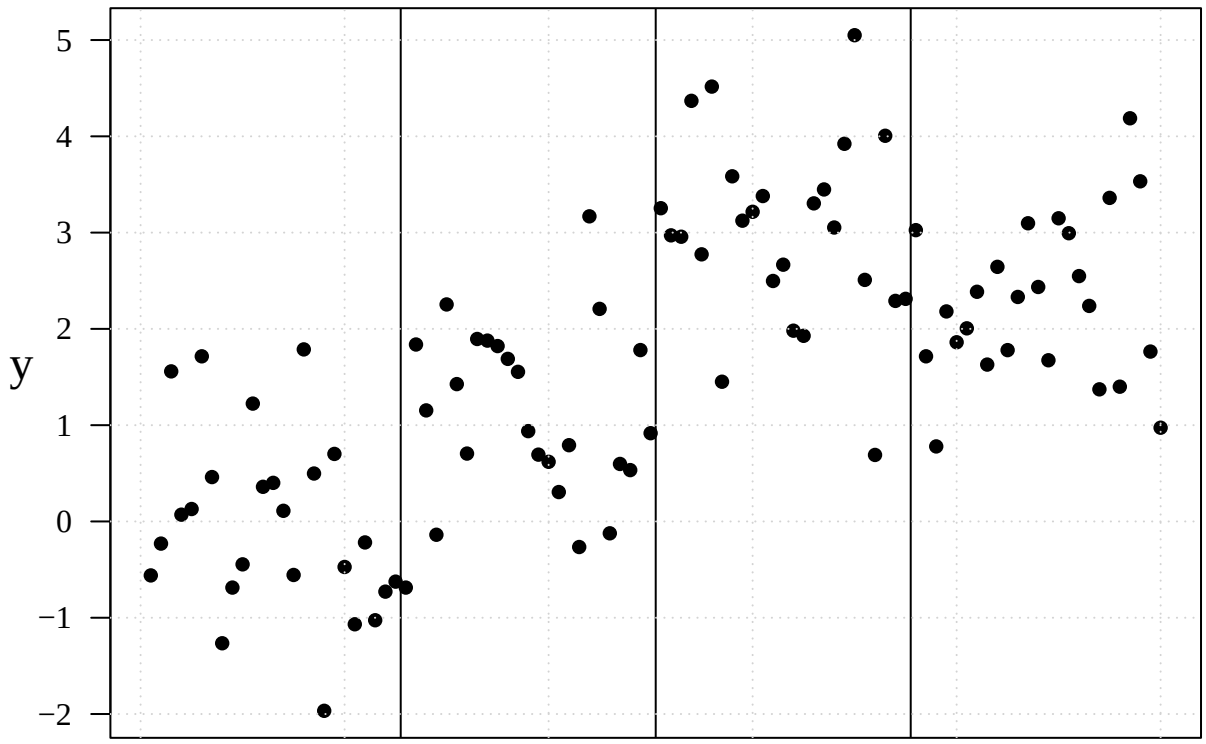
$$x_{ij} = \mu_i + \varepsilon_{ij}, \qquad i = 1, \ldots, k, \quad j = 1, \ldots, n_i,$$

where μ_i is the mean of group i, and the errors ε_{ij} are assumed to be independent with mean 0 and a common variance σ^2. Our goal is to compare the group means

$$\mu_1, \mu_2, \ldots, \mu_k.$$

To illustrate, we create four groups of 25 observations with population means 0, 1, 3, and 2. We use the random errors `y` generated in the previous example.

```
means <- rep(c(0, 1, 3, 2), each = 25) # Repeat each group mean 25 times
y1 <- y + means                        # Add random errors to the group means

par(mar = c(3, 3.5, 0.5, 0.6), mgp = c(2, 1, 0), las = 1, family = "serif") # Set plot appearance
plot(y1, pch = 16, ylab = "", xaxt = "n", cex.lab = 1.5) # Plot all 100 observations
mtext(expression(y), 2, line = 2, cex = 1.5)             # Add y-axis label
abline(v = c(25.5, 50.5, 75.5))                          # Separate the four groups
grid()
```



Index

The plot shows four groups, each containing 25 observations. The differences in their vertical locations reflect differences among the group means, while the variation of observations within each group reflects the common error variance σ^2.

Because σ^2 is unknown, ANOVA estimates it by pooling the **within-group variation** across all k groups. The resulting estimator is the **Mean Squared Error (MSE)**:

$$\hat{\sigma}^2 = MSE = \frac{SS_W}{N-k},$$

where SS_W is the within-group sum of squares, $N = \sum_{i=1}^{k} n_i$ is the total sample size, and $N-k$ is the error degrees of freedom.

For group i, the sample mean $\bar{x}_i$ estimates μ_i. Under the common-variance assumption, its standard error is

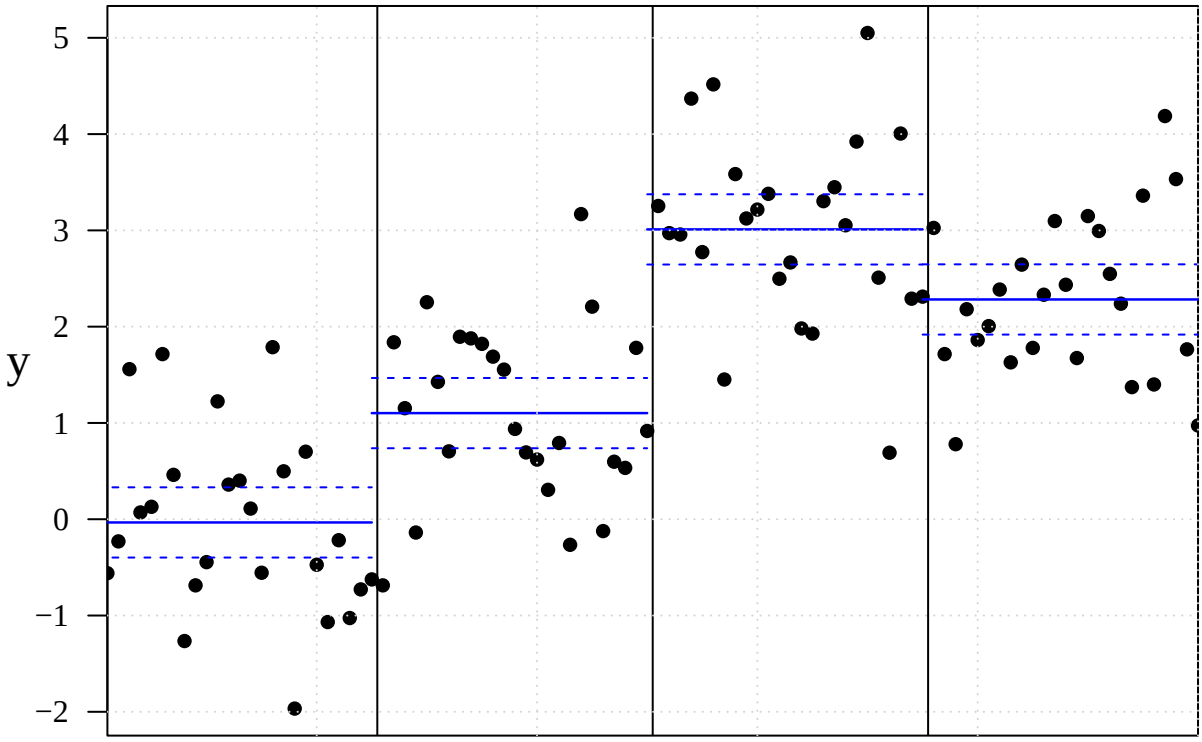
$$SE(\bar{x}_i) = \sqrt{\frac{MSE}{n_i}}.$$

Thus, a $(1-\alpha)100\%$ confidence interval for the group mean μ_i is

$$\bar{x}_i \ \pm \ t_{N-k,\,1-\alpha/2}\sqrt{\frac{MSE}{n_i}}.$$

Compared with the one-sample t interval, the key difference is that the unknown variance σ^2 is now estimated by **pooling information across all groups** through the ANOVA MSE.

```r
dat <- array(y1, dim = c(25, 4))              # Reshape the 100 observations into 4 groups
means <- apply(dat, 2, mean)                  # Compute the sample mean for each group
group_var <- apply(dat, 2, var)               # Sample variance within each group
MSE <- sum((25 - 1) * group_var) / (100 - 4) # Pooled within-group variance
se <- sqrt(MSE / 25)                          # Standard error of each group mean

par(mar = c(3, 3.5, 0.5, 0.6), mgp = c(2, 1, 0), las = 1, family = "serif") # Set plot appearance
plot(y1, pch = 16, ylab = "", xaxt = "n", cex.lab = 1.5, xaxs = "i")  # Plot all observations
mtext(expression(y), 2, line = 2, cex = 1.5)                          # Add y-axis label
abline(v = c(25.5, 50.5, 75.5))                                       # Separate the four groups

for (i in 1:4){
  segments((i - 1) * 25, means[i], 25 * i, col = "blue", lwd = 1.25) # Group sample mean
  segments((i - 1) * 25, means[i] + qt(0.975, 96) * se, 25 * i, col = "blue", lty = 2) # Upper 95% CI
  segments((i - 1) * 25, means[i] - qt(0.975, 96) * se, 25 * i, col = "blue", lty = 2) # Lower 95% CI
}
grid() # Add reference grid
```



Index

We reshape the data into four groups, compute each group mean, and estimate the common standard error using the pooled within-group variance from ANOVA. The plot then adds the estimated group means and their 95% confidence limits.

Inference for Regression Function (Simple Linear Regression)

We now extend the idea of estimating a mean to a setting where the **population mean changes with a (numerical) predictor** x. In simple linear regression, we assume

$$Y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \qquad \varepsilon_i \overset{\text{i.i.d.}}{\sim} N(0, \sigma^2).$$

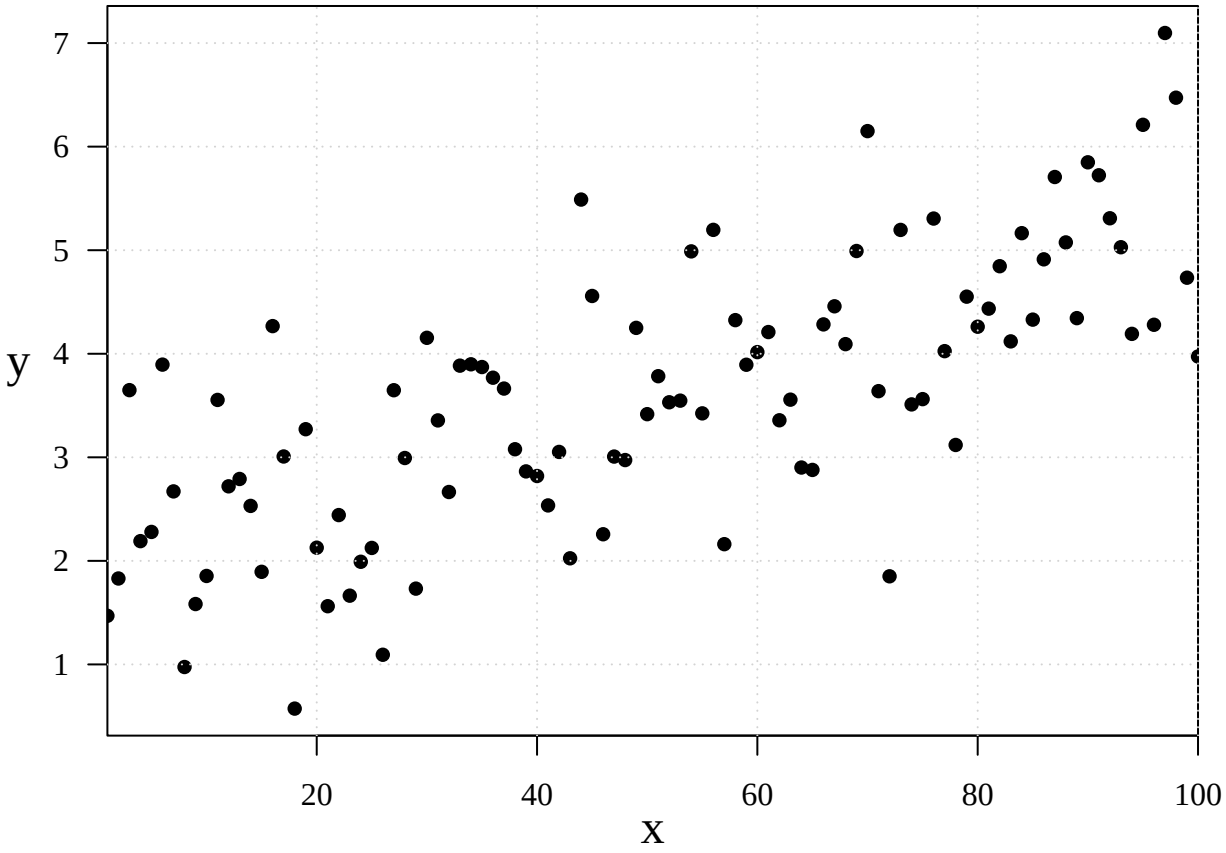
The conditional mean of Y at a given value x is therefore

$$E(Y \mid x) = \beta_0 + \beta_1 x.$$

Our goal is to estimate this **regression function** and quantify the uncertainty in the estimated mean response.

To illustrate, we simulate observations from a linear model with intercept $\beta_0 = 2$, slope $\beta_1 = 0.03$, and the random errors `y` generated earlier.

```
x <- 1:100                    # Predictor values from 1 to 100
y2 <- 2 + 0.03 * x + y        # Response = regression function + random error

par(mar = c(3, 3.5, 0.5, 0.6), mgp = c(2, 1, 0), las = 1, family = "serif") # Set plot appearance
plot(y2 ~ x, pch = 16, ylab = "", cex.lab = 1.5, xaxs = "i") # Scatterplot of response versus predictor
mtext(expression(y), 2, line = 2, cex = 1.5)                 # Add y-axis label
grid()                                                        # Add reference grid
```



The scatterplot shows a positive linear relationship between x and Y, together with random variation around the underlying regression line.

For a particular predictor value x_0, the estimated mean response is

$$\hat{y}_0 = \hat{\beta}_0 + \hat{\beta}_1 x_0.$$

A $(1-\alpha)100\%$ confidence interval for the **mean response** $E(Y \mid x_0)$ is

$$\hat{y}_0 \pm t_{n-2,\,1-\alpha/2}\, s \sqrt{\frac{1}{n} + \frac{(x_0 - \bar{x})^2}{\sum_{i=1}^{n}(x_i - \bar{x})^2}},$$

where s estimates the error standard deviation σ.

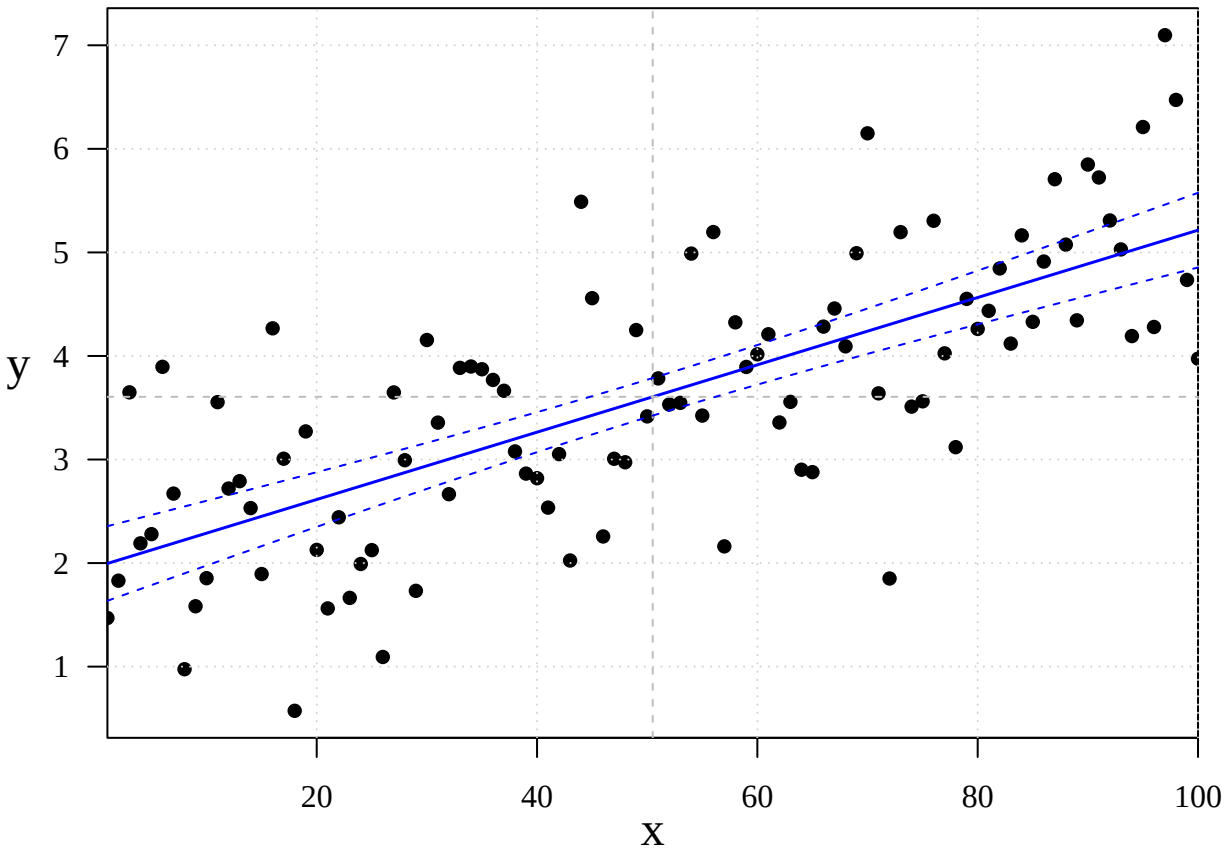
Notice that the standard error depends on x_0. It is smallest near $\bar{x}$ and increases as x_0 moves farther from the center of the observed predictor values.

We use `lm()` to fit the regression model and `predict()` to obtain the fitted regression function and its 95% confidence band.

```
fit <- lm(y2 ~ x) # Fit the simple linear regression model
summary(fit)      # Display estimated coefficients and model summary
```

```
##
## Call:
## lm(formula = y2 ~ x)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.45356 -0.55236 -0.03462  0.64850  2.09487
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 1.963596   0.184287   10.65   <2e-16 ***
## x           0.032511   0.003168   10.26   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9145 on 98 degrees of freedom
## Multiple R-squared:  0.518,  Adjusted R-squared:  0.513
## F-statistic: 105.3 on 1 and 98 DF,  p-value: < 2.2e-16
```

```
x_grid <- data.frame(x = seq(1, 100, length.out = 1000)) # Fine grid of x values for plotting
CI_band <- predict(fit, newdata = x_grid, interval = "confidence") # 95% CI for mean response

par(mar = c(3, 3.5, 0.5, 0.6), mgp = c(2, 1, 0), las = 1, family = "serif") # Set plot appearance
plot(y2 ~ x, pch = 16, ylab = "", cex.lab = 1.5, xaxs = "i") # Plot observations
mtext(expression(y), 2, line = 2, cex = 1.5)                 # Add y-axis label
grid()                                                        # Add reference grid
abline(fit, col = "blue", lwd = 1.5)                         # Fitted regression line
abline(v = mean(x), lty = 2, col = "gray")                   # Mean of x
abline(h = mean(y2), lty = 2, col = "gray")                  # Mean of y
lines(x_grid$x, CI_band[, "lwr"], lty = 2, col = "blue")     # Lower 95% confidence limit
lines(x_grid$x, CI_band[, "upr"], lty = 2, col = "blue")     # Upper 95% confidence limit
```



Time Series Data

Lake Huron Time Series

We now move from independent observations to **time series data**, where observations are recorded sequentially over time and their ordering matters.

The built-in R dataset `LakeHuron` contains annual measurements of the **level of Lake Huron (in feet)** from 1875 to 1972. We begin by loading the data and examining its time-series structure.

```
data(LakeHuron) # Load the built-in Lake Huron time series
LakeHuron       # Display the observations
```

```
## Time Series:
## Start = 1875
## End = 1972
## Frequency = 1
##  [1] 580.38 581.86 580.97 580.80 579.79 580.39 580.42 580.82 581.40 581.32
## [11] 581.44 581.68 581.17 580.53 580.01 579.91 579.14 579.16 579.55 579.67
## [21] 578.44 578.24 579.10 579.09 579.35 578.82 579.32 579.01 579.00 579.80
## [31] 579.83 579.72 579.89 580.01 579.37 578.69 578.19 578.67 579.55 578.92
## [41] 578.09 579.37 580.13 580.14 579.51 579.24 578.66 578.86 578.05 577.79
## [51] 576.75 576.75 577.82 578.64 580.58 579.48 577.38 576.90 576.94 576.24
## [61] 576.84 576.85 576.90 577.79 578.18 577.51 577.23 578.42 579.61 579.05
## [71] 579.26 579.22 579.38 579.10 577.95 578.12 579.75 580.85 580.41 579.96
## [81] 579.61 578.76 578.18 577.21 577.13 579.10 578.25 577.91 576.89 575.96
```

```
## [91] 576.80 577.68 578.38 578.52 579.74 579.31 579.89 579.96
```

```
start(LakeHuron); end(LakeHuron); frequency(LakeHuron) # Check time range and sampling frequency
```

```
## [1] 1875    1
```

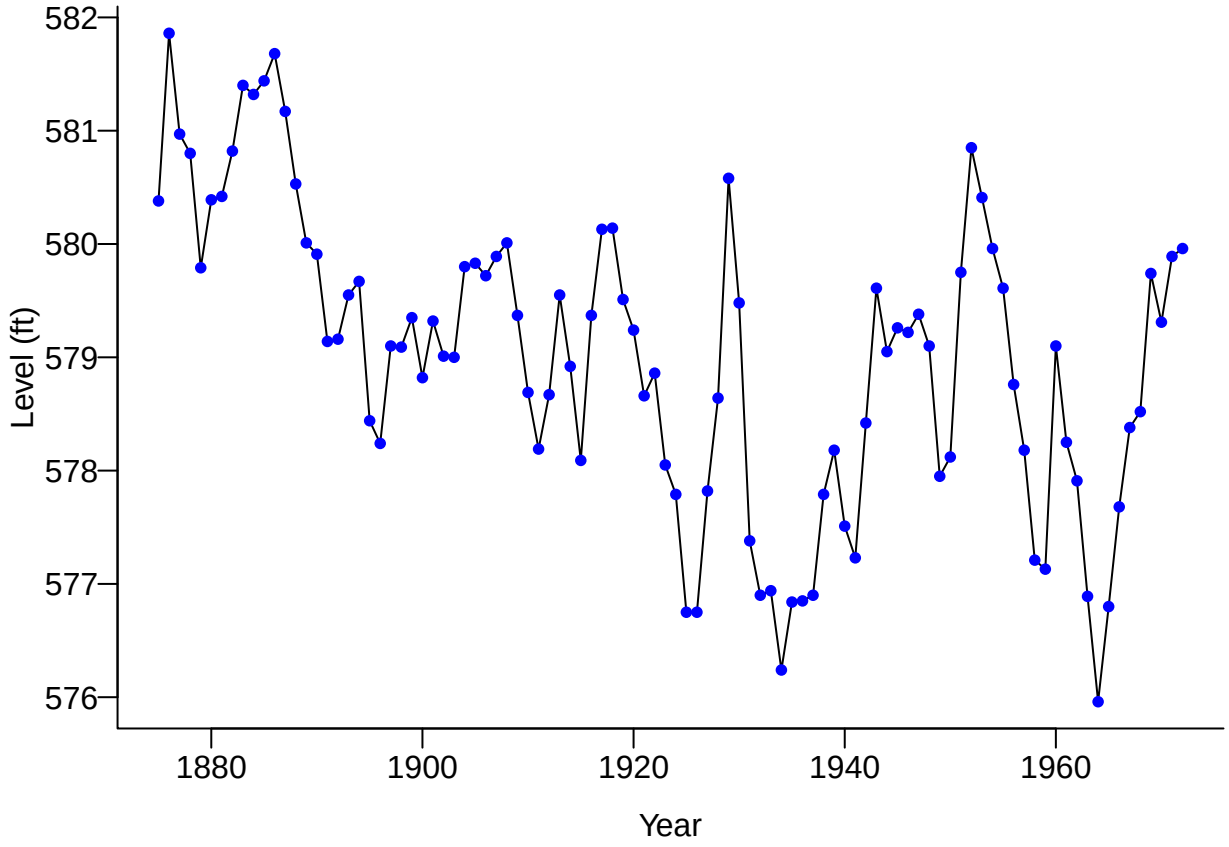
```
## [1] 1972    1
```

```
## [1] 1
```

In R, a time series can be stored as a `ts` object. Such an object contains not only the observed values but also information about **when the series begins, when it ends, and how frequently observations are recorded**. Here, `frequency(LakeHuron)` returns 1 because there is one observation per year.

A natural first step in any time series analysis is to **plot the observations against time**.

```
par(mar = c(3.2, 3.2, 0.5, 0.5), mgp = c(2, 0.5, 0), bty = "L") # Set plot margins and axis appearance
plot(LakeHuron, ylab = "Level (ft)", xlab = "Year", las = 1) # Plot lake level against year
points(LakeHuron, cex = 0.8, col = "blue", pch = 16)         # Highlight the annual observations
```



Unlike an ordinary sample, the horizontal axis now has a meaningful ordering: each observation occurs **after the previous observation in time**. The plot allows us to begin looking for important time-series features such as **changes in level, persistent patterns, and temporal dependence**.

This leads to a central question in time series analysis:

How does the behavior of an observation depend on what happened in the past?

CO_2 Concentration

The built-in R dataset `co2` contains **monthly atmospheric CO_2 concentrations**, measured in parts per million (ppm). Unlike the annual Lake Huron series, these data are observed monthly, allowing us to see variation both **across years and within each year**.

```
data(co2) # Load the built-in monthly CO2 time series
start(co2); end(co2); frequency(co2) # Check time range and frequency (12 observations per year)
```

```
## [1] 1959    1
```

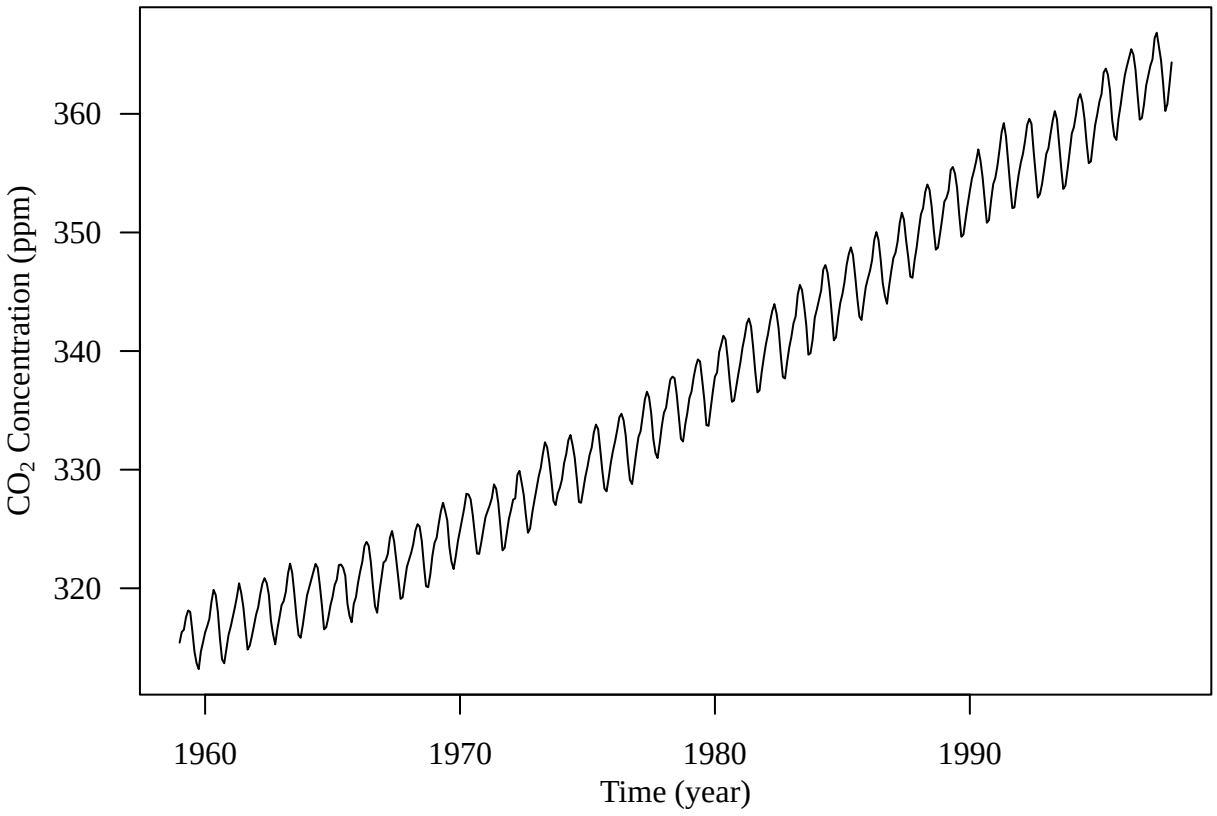
```
## [1] 1997   12
```

```
## [1] 12
```

The `frequency()` function returns 12, indicating that the series contains **12 observations per year**.

We next plot CO_2 concentration against time.

```
par(mar = c(3.8, 4, 0.8, 0.6), family = "serif") # Set plot margins
plot(co2, las = 1, xlab = "", ylab = "") # Plot monthly CO2 concentrations over time
mtext("Time (year)", side = 1, line = 2) # Add x-axis label
mtext(expression(paste("CO"[2], " Concentration (ppm)")), side = 2, line = 2.5) # Add y-axis label
```



Two important time-series features are immediately visible:

- **Trend:** CO_2 concentrations increase systematically over time.

- **Seasonality:** a regular pattern repeats approximately every 12 months.

Thus, the observed series contains structure operating at **different time scales**: a long-term change in level together with shorter-term seasonal fluctuations.

This motivates an important idea in time series analysis: before modeling the remaining dependence, we often need to identify and account for systematic components such as **trend and seasonality**.

Apple Daily Log Returns

Financial time series provide another type of temporal structure. Rather than analyzing the stock price itself, we often work with **returns**, which measure relative changes in price over time.

For a closing price P_t, the daily log return is

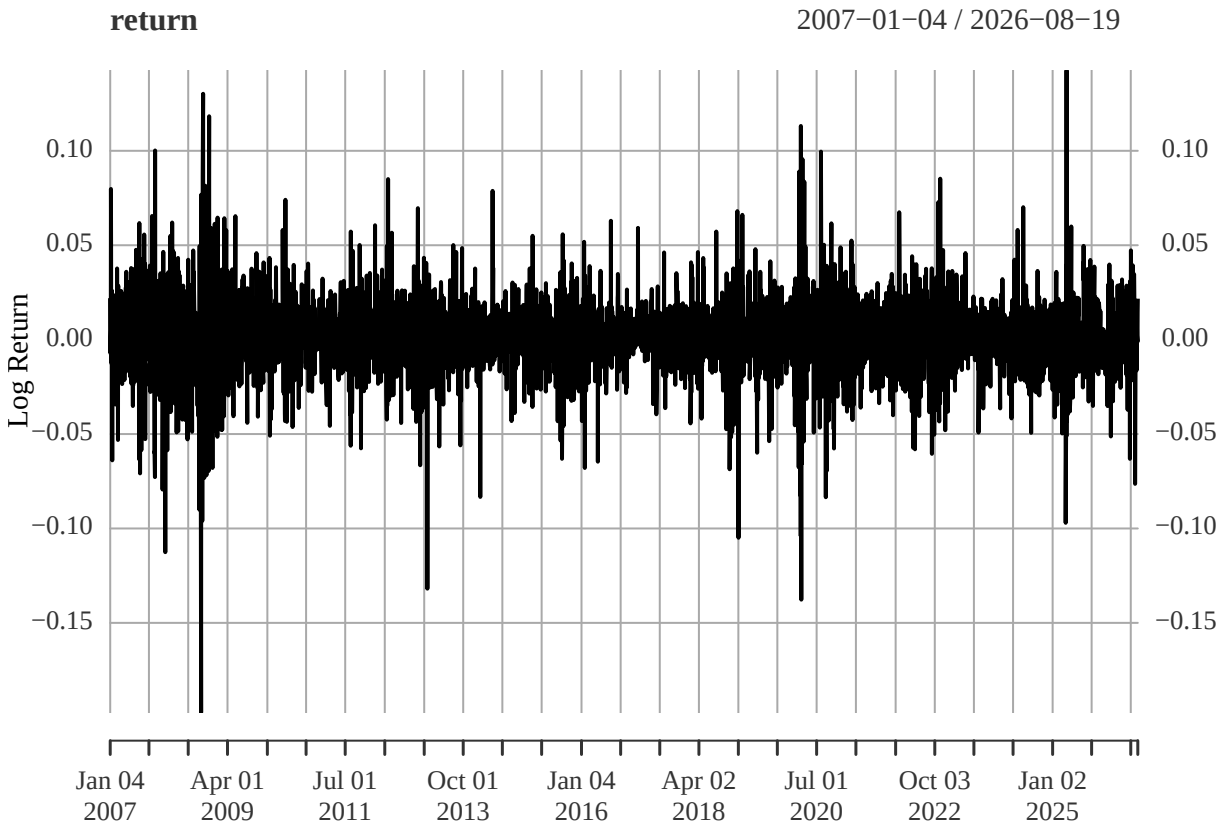
$$r_t = \log(P_t) - \log(P_{t-1}) = \log\left(\frac{P_t}{P_{t-1}}\right).$$

We use the `quantmod` package to download daily Apple (AAPL) stock prices from Yahoo Finance and compute the corresponding log returns.

```
library(quantmod) # Load tools for downloading and working with financial data
getSymbols("AAPL", src = "yahoo") # Download Apple stock-price data from Yahoo Finance
```

```
## [1] "AAPL"
```

```
closing <- AAPL$AAPL.Close # Extract daily closing prices
return <- diff(log(closing)) # Compute daily log returns
return <- na.omit(return) # Remove the missing value created by differencing

par(las = 1, mgp = c(2.2, 1, 0), mar = c(3.6, 3.6, 0.8, 0.6), family = "serif") # Set plot appearance
plot(return, xlab = "Date", ylab = "Log Return") # Plot daily log returns over time
```



Unlike the CO_2 series, the returns fluctuate around a relatively stable level and do not exhibit an obvious long-term trend or seasonal pattern. However, the **magnitude of the fluctuations changes over time**.

Periods of large movements tend to be followed by other large movements, while relatively calm periods tend to persist. This phenomenon is known as **volatility clustering**.

Thus, even when the level of a time series appears relatively stable, its **variability may still have temporal structure**.

Later in the course, we will study **ARCH/GARCH models**, which are designed specifically to model this type of time-varying conditional variance.

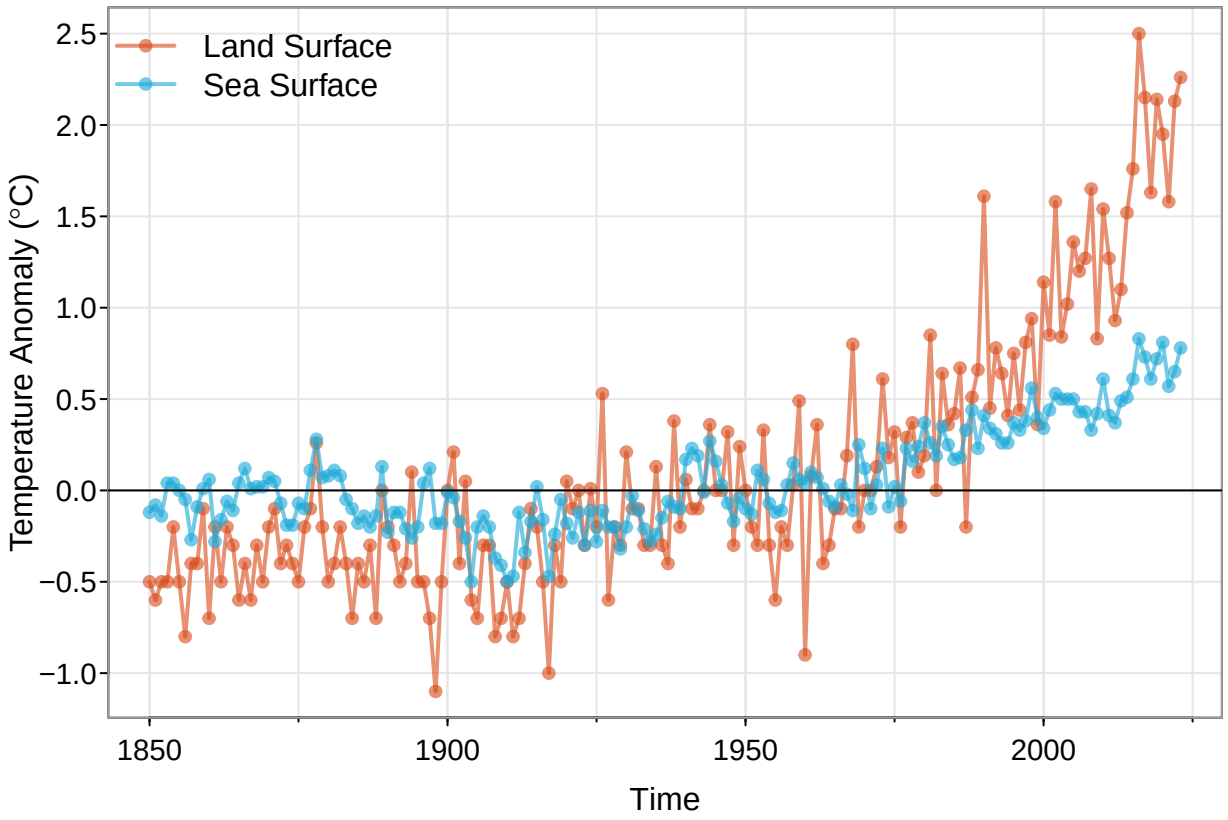
Global Mean Land and Ocean Temperature Anomalies

So far, we have considered **one time series at a time**. In many applications, however, several related variables are observed over the same time period. Together, these form a **multivariate time series**.

The `astsa` package contains annual global temperature anomaly series for both **land surface** and **sea surface** temperatures. We plot the two series together to examine how they evolve jointly over time.

```
library(astsa) # Load time-series datasets and functions from the astsa package

culer <- c(rgb(217, 77, 30, 160, max = 255), rgb(30, 170, 217, 160, max = 255))
# Define colors for the two series
tsplot(gtemp_land, col = culer[1], lwd = 2, type = "o", pch = 20, las = 1,
       ylab = expression(paste("Temperature Anomaly (", degree, "C)")))
# Plot land temperature anomalies
lines(gtemp_ocean, col = culer[2], lwd = 2, type = "o", pch = 20) # Add ocean temperature anomalies
abline(h = 0) # Add reference line for zero anomaly
```

```r
legend("topleft", col = culer, lty = 1, lwd = 2, pch = 20,
       legend = c("Land Surface", "Sea Surface"), bg = "white", bty = "n") # Add legend
```



The two series exhibit a clear long-term warming pattern and tend to move together. At the same time, their year-to-year fluctuations and magnitudes are not identical.

This illustrates the key idea of a **multivariate time series**: we are interested not only in the temporal behavior of each variable,

$$Y_{1,t}, \quad Y_{2,t},$$

but also in how the variables are related **to each other over time**. We can represent the observations at time t as a vector,

$$\mathbf{Y}_t = \begin{pmatrix} Y_{1,t} \\ Y_{2,t} \end{pmatrix} = \begin{pmatrix} \text{land temperature anomaly} \\ \text{ocean temperature anomaly} \end{pmatrix}.$$

Later, multivariate time-series methods allow us to model both **temporal dependence within each series** and **dependence across multiple series**.

Simulated Time Series: Random Walk and Brownian Motion

Our final example is a **simulated stochastic process** rather than an observed dataset. Consider the random walk

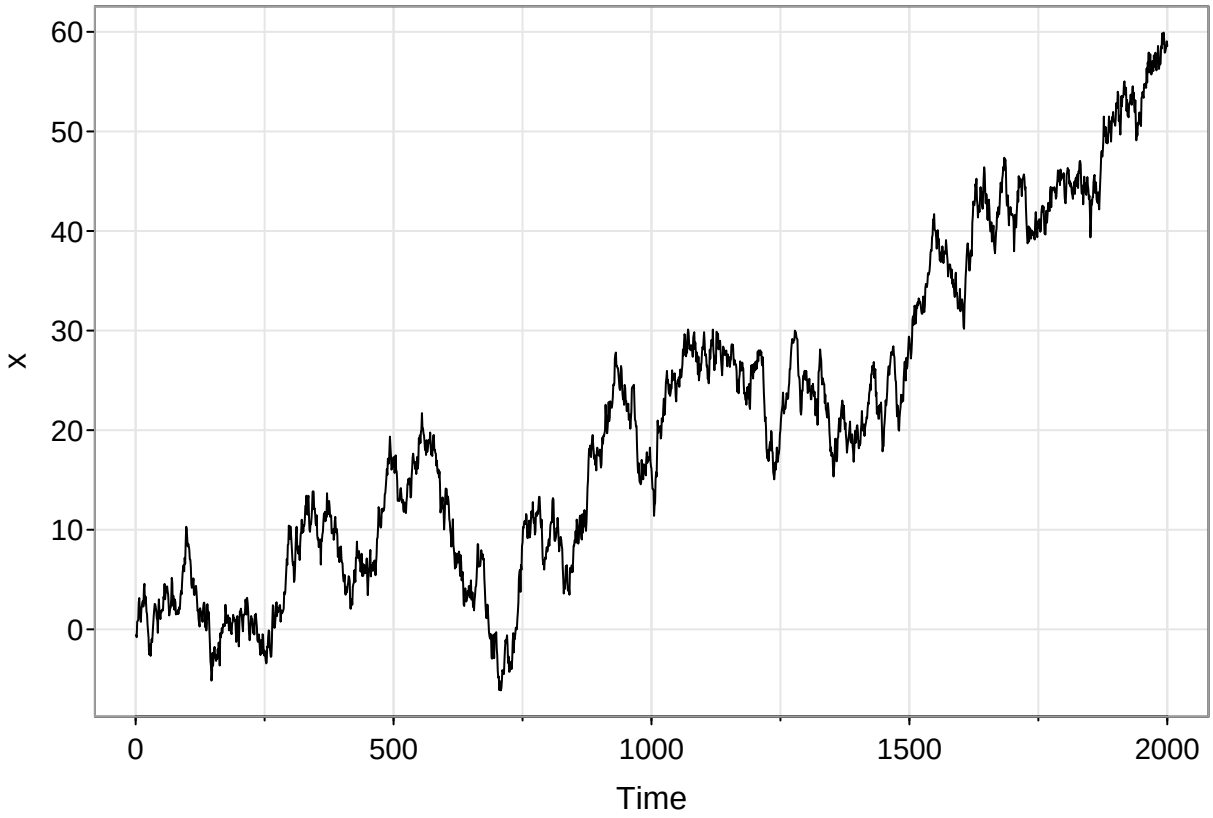
$$X_t = X_{t-1} + W_t, \qquad W_t \overset{\text{i.i.d.}}{\sim} N(0,1),$$

or equivalently,

$$X_t = \sum_{j=1}^{t} W_j.$$

Each new value is obtained by adding a random increment to the previous value.

```r
set.seed(123)            # Make the simulation reproducible
w <- rnorm(2000)         # Generate 2000 independent N(0,1) random increments
x <- cumsum(w)           # Cumulative sums produce the random walk
tsplot(x, las = 1)       # Plot the simulated time series
```



Although the increments W_t are independent, the values of X_t are **strongly dependent over time** because each value contains all previous random increments. Consequently, the series can exhibit long apparent upward or downward movements even though the increments have mean zero.

This discrete random walk is also closely related to **Brownian motion**, a fundamental continuous-time stochastic process. If we use increasingly small time steps and appropriately scale the random increments,

$$X(t + \Delta t) - X(t) \sim N(0, \Delta t),$$

the resulting random walk provides a discrete approximation to Brownian motion.

Thus, time series can arise not only from observations collected at naturally discrete times, but also as **discrete measurements or approximations of processes evolving continuously in time**.

What Is a Time Series?

A **time series** is a collection of observations

$$\{y_t : t \in T\},$$

recorded sequentially over time, where T denotes the set of observation times. A defining feature of time-series data is that **the temporal ordering of the observations matters**.

The time index T can take several forms:

- **Equally spaced discrete time:** $T = \{0, 1, 2, \ldots\}$, such as daily, monthly, or annual observations.
- **Irregularly spaced discrete time:** observations are recorded at unequal time intervals.
- **Continuous time:** T is an interval, such as $T = [0, \tau]$, and the process evolves continuously over time.

Different sampling structures generally require different statistical methods. In this course, we will focus primarily on **equally spaced discrete-time series**, while briefly connecting them to continuous-time and spatio-temporal processes later in the course.

How Do Discrete-Time Series Arise?

Some time series are **intrinsically discrete in time**. For example, we might record

- the number of flights departing ATL each day, or
- daily sales of a product.

Other discrete-time series arise by observing or summarizing an underlying **continuous-time process**. Common mechanisms include:

- **Sampling:** recording the process at selected times, such as wind speed measured every hour;
- **Aggregation:** accumulating values over an interval, such as total daily precipitation;
- **Extrema:** recording an extreme over an interval, such as daily maximum temperature.

Thus, even when our observed data are discrete in time, the underlying physical process may evolve continuously.

Continuous-time models can also be particularly useful when observations occur at **irregular time points**.

What Values Can a Time Series Take?

The observations y_t need not be continuous numerical measurements. Depending on the application, a time series may be:

- **Real-valued:** values in $\mathbb{R}$ or a subset thereof, such as temperature or stock returns;
- **Count-valued:** non-negative integers, such as the number of events occurring during a time interval (see Jia et al. (2021) for a recent development in modeling count time series);
- **Categorical:** for example, the outcome of a scheduled basketball game: win, loss, or cancellation;
- **Circular:** angular measurements such as wind direction (see Breckling (2012));
- **Complex-valued:** encountered in some signal-processing and electrical-engineering applications.

The type of values taken by y_t plays an important role in determining which time-series models and methods are appropriate.

Exploratory Time Series Analysis

Given observations $\{y_t : t \in T\}$, the first step is usually to **plot y_t against time**.

A well-designed time-series plot should clearly identify the **time scale** on the horizontal axis and the **variable and its units** on the vertical axis.

Before fitting a formal model, we use exploratory analysis to ask:

1. **Trend:** Is there a systematic long-term increase or decrease?
2. **Seasonality:** Are there patterns that repeat at regular time intervals?
3. **Changes in behavior:** Are there abrupt shifts in the mean, variance, or other characteristics of the series?
4. **Outliers:** Are there unusual observations that require further investigation?
5. **Changing variability:** Does the magnitude of the fluctuations change over time?
6. **Transformation:** Would a transformation help stabilize the variance or simplify the structure?
7. **Temporal dependence:** Do observations close together in time tend to behave similarly?

These features help determine what type of time-series model may be appropriate.

Trend, Seasonality, and Noise

A useful way to describe many time series is to separate their systematic structure into **trend**, **seasonality**, and a remaining **noise process**.

Trend

A trend, μ_t, represents a **smooth, long-term change in the level** of a time series, typically through its mean. The key idea is that a trend varies gradually over time rather than from one observation to the next.

The form of μ_t is generally unknown and must be specified or estimated from the data. After estimating and removing the trend, we obtain a detrended series.

Seasonal Component

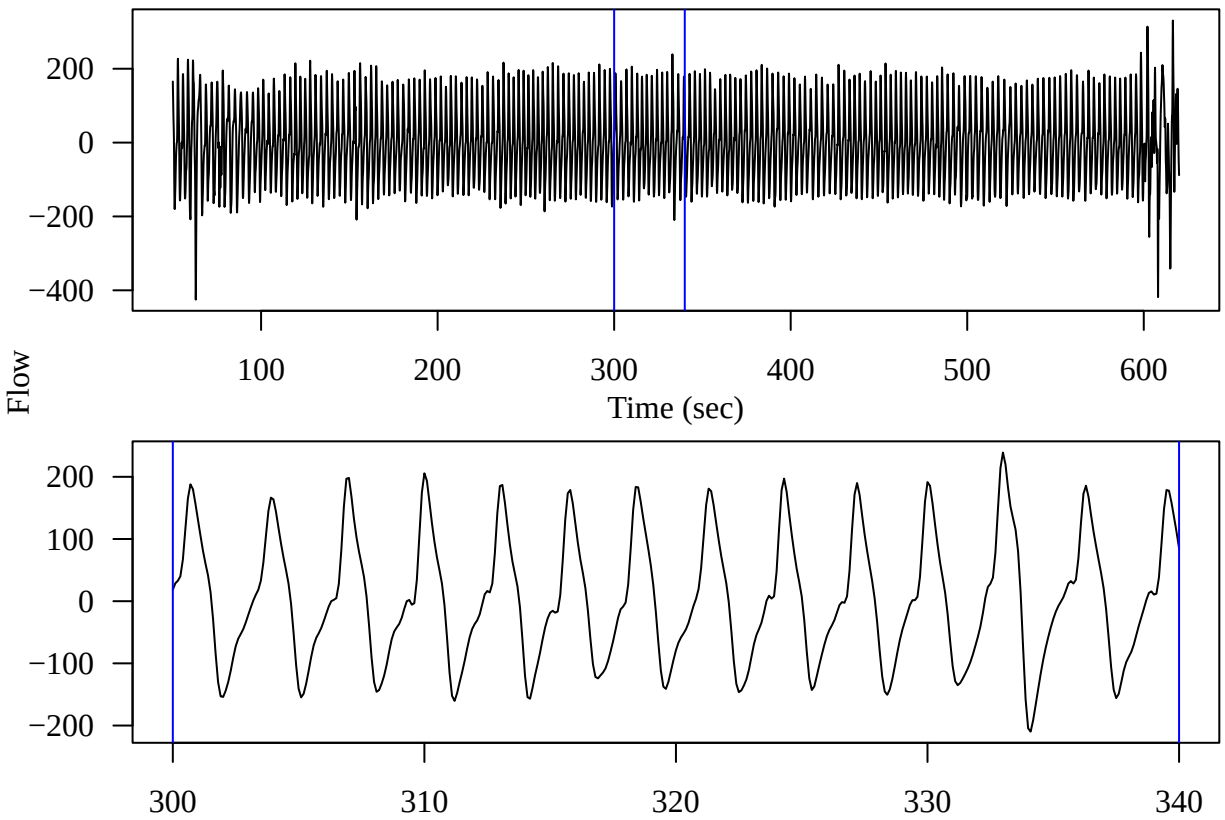
A seasonal component, s_t, represents a pattern that **repeats at regular time intervals**. For a seasonal period d,

$$s_t = s_{t+kd}, \qquad k \in \mathbb{Z}.$$

Seasonality may have a simple structure, such as a sine or cosine wave, or a much more complex repeating pattern. The sleep airflow data in Huang et al. (2022) provide an example of a complex periodic structure.

```
load("Flow_sub1.RData") # Load the sleep airflow data
par(mfrow = c(2, 1), mar = c(2.6, 3.6, 0.8, 0.6), family = "serif") # Display two plots vertically

id <- 500:6200 # Select a longer segment of the airflow series
plot(id / 10, flow[id], type = "l", las = 1, xlab = "", ylab = "") # Plot airflow over time
```

```r
mtext("Time (sec)", 1, line = 2); mtext("Flow", 2, line = 2.5, at = -650) # Add axis labels
abline(v = c(3000, 3400) / 10, col = "blue") # Mark the interval enlarged below

id2 <- 3000:3400 # Select the highlighted interval
plot(id2 / 10, flow[id2], type = "l", las = 1, xlab = "", ylab = "") # Examine its repeating structure
abline(v = c(3000, 3400) / 10, col = "blue")
```



The upper panel shows a longer portion of the airflow signal, while the lower panel magnifies a shorter interval. At the finer scale, the repeating breathing cycles become much easier to see.

Noise

After accounting for trend and seasonality, the remaining component is the **noise process**, η_t. Importantly, "noise" does not necessarily mean independent or completely random variation. It may still exhibit substantial **temporal dependence**.

A common decomposition is therefore

$$Y_t = \mu_t + s_t + \eta_t,$$

where μ_t represents trend, s_t represents seasonality, and η_t represents the remaining stochastic variation.

A major focus of this course will be to identify plausible statistical models for η_t, typically beginning with **stationary processes**.

Combining Trend, Seasonality, and Noise

Trend, seasonality, and noise can be combined in different ways. Two common representations are:

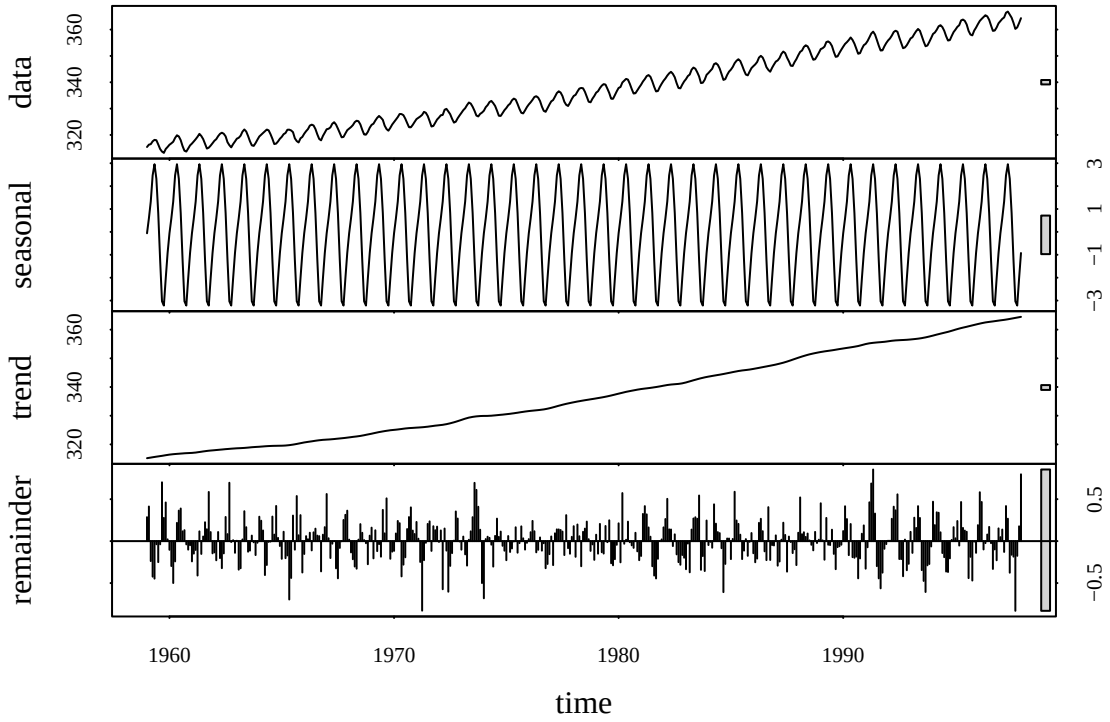
1. **Additive decomposition**

$$y_t = \mu_t + s_t + \eta_t, \qquad t = 1, \ldots, T.$$

The additive model is appropriate when the **magnitude of the seasonal fluctuations is roughly constant over time**, regardless of the level of the series.

For example, we can use **STL (Seasonal-Trend decomposition using Loess)** to decompose the CO_2 series into trend, seasonal, and remainder components.

```
par(mar = c(4, 3.6, 0.8, 0.6), family = "serif") # Set plot margins
stl_fit <- stl(co2, s.window = "periodic")
# Decompose CO2 into seasonal, trend, and remainder components
plot(stl_fit, las = 1) # Plot the original series and its decomposition
```



Here, `s.window = "periodic"` assumes that the seasonal pattern repeats consistently from year to year. The **remainder** is what remains after removing the estimated trend and seasonal components.

2. **Multiplicative decomposition**

$$y_t = \mu_t s_t \eta_t, \qquad t = 1, \ldots, T.$$

A multiplicative representation is useful when the **size of the seasonal fluctuations changes with the level of the series**. For example, seasonal variation may become larger as the overall level increases.

When all components are positive, taking logarithms converts the multiplicative structure into an additive one:

$$\log(y_t) = \log(\mu_t) + \log(s_t) + \log(\eta_t).$$

This is one reason logarithmic transformations are common in time series analysis: they can simplify multiplicative structure and help stabilize variability.

When results are obtained on a transformed scale, remember that they may need to be **transformed back to the original measurement scale** for interpretation.

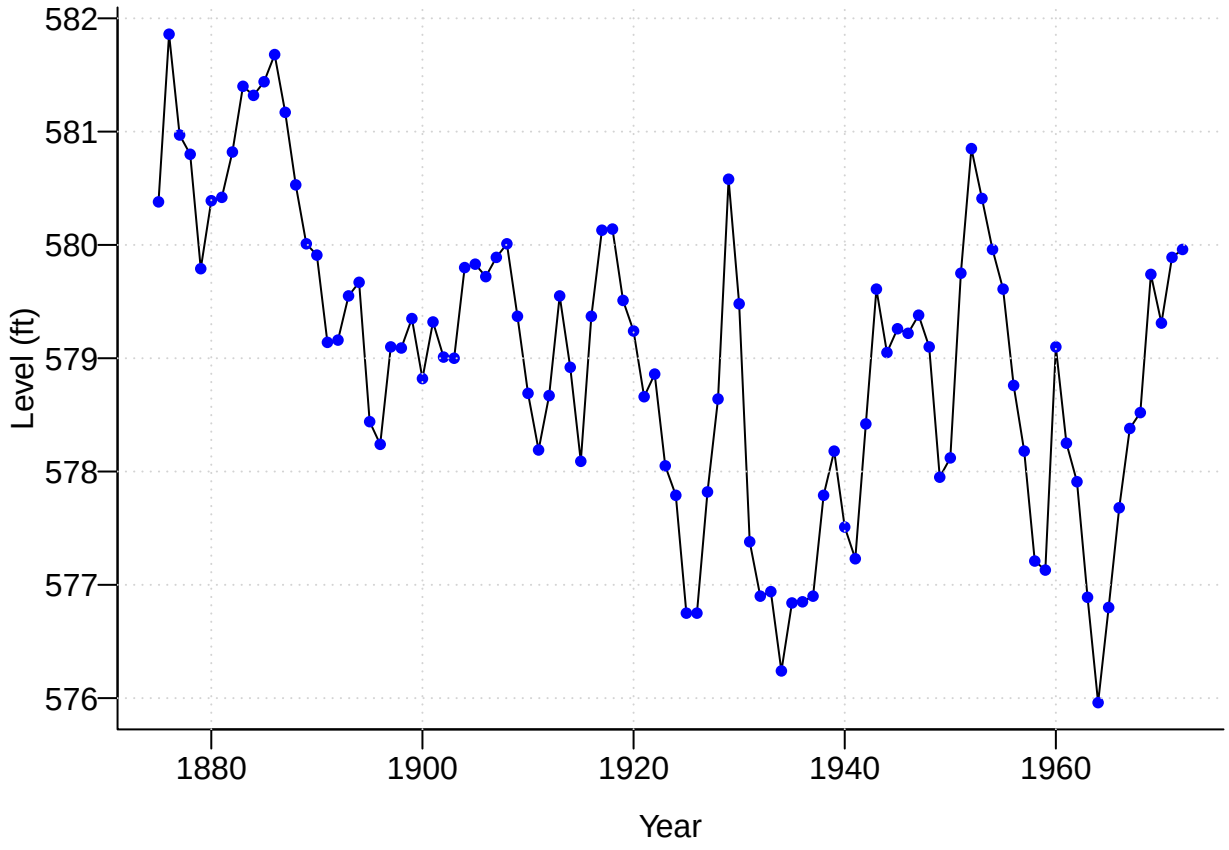
Lake Huron Time Series Example

This example is adapted from Brockwell et al. (2002). The `LakeHuron` dataset contains annual measurements of the **level of Lake Huron**, in feet, from 1875 to 1972.

```
data(LakeHuron); str(LakeHuron) # Load the data and examine its structure
```

```
##  Time-Series [1:98] from 1875 to 1972: 580 582 581 581 580 ...
```

```
par(mar = c(3.2, 3.2, 0.5, 0.5), mgp = c(2, 0.5, 0), bty = "L") # Set plot appearance
plot(LakeHuron, ylab = "Level (ft)", xlab = "Year", las = 1) # Plot lake level over time
points(LakeHuron, cex = 0.8, col = "blue", pch = 16) # Highlight annual observations
grid() # Add reference grid
```



This is a **real-valued, equally spaced, discrete-time series**, with one observation per year.

The plot suggests two important features:

1. a long-term **decreasing trend**; and
2. shorter-term **fluctuations around the trend**.

A natural first step is to separate these two components. As a simple approximation, suppose the trend is linear:

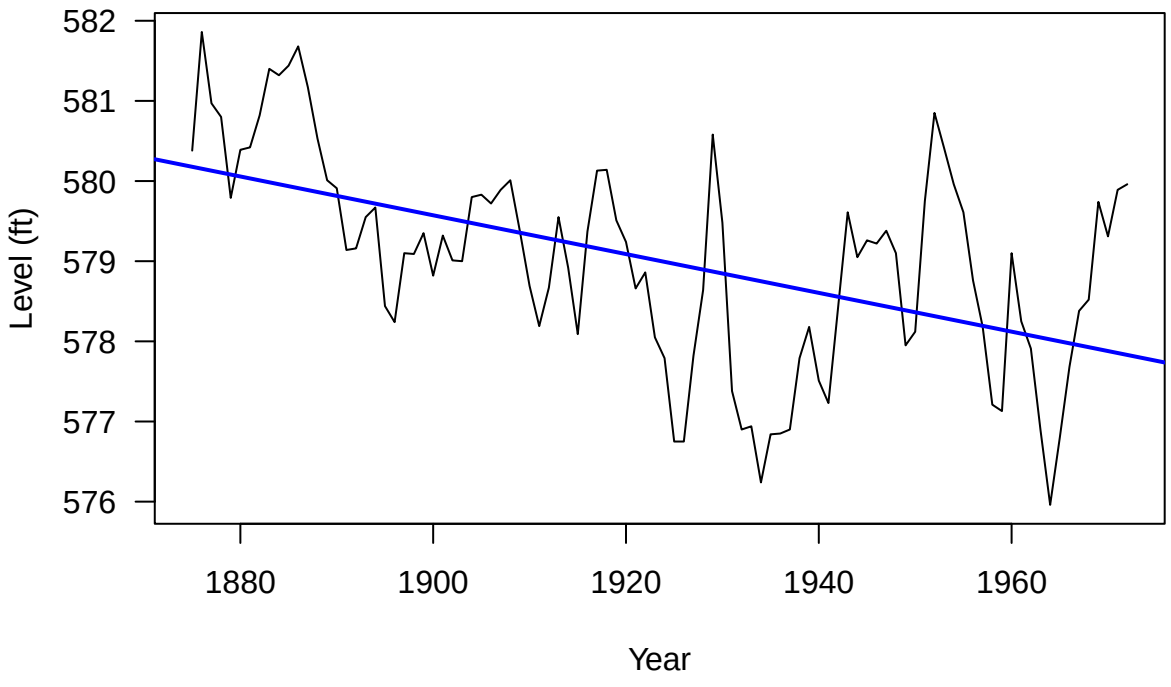
$$Y_t = \beta_0 + \beta_1 t + \eta_t,$$

where $\beta_0 + \beta_1 t$ represents the long-term trend and η_t represents the remaining variation.

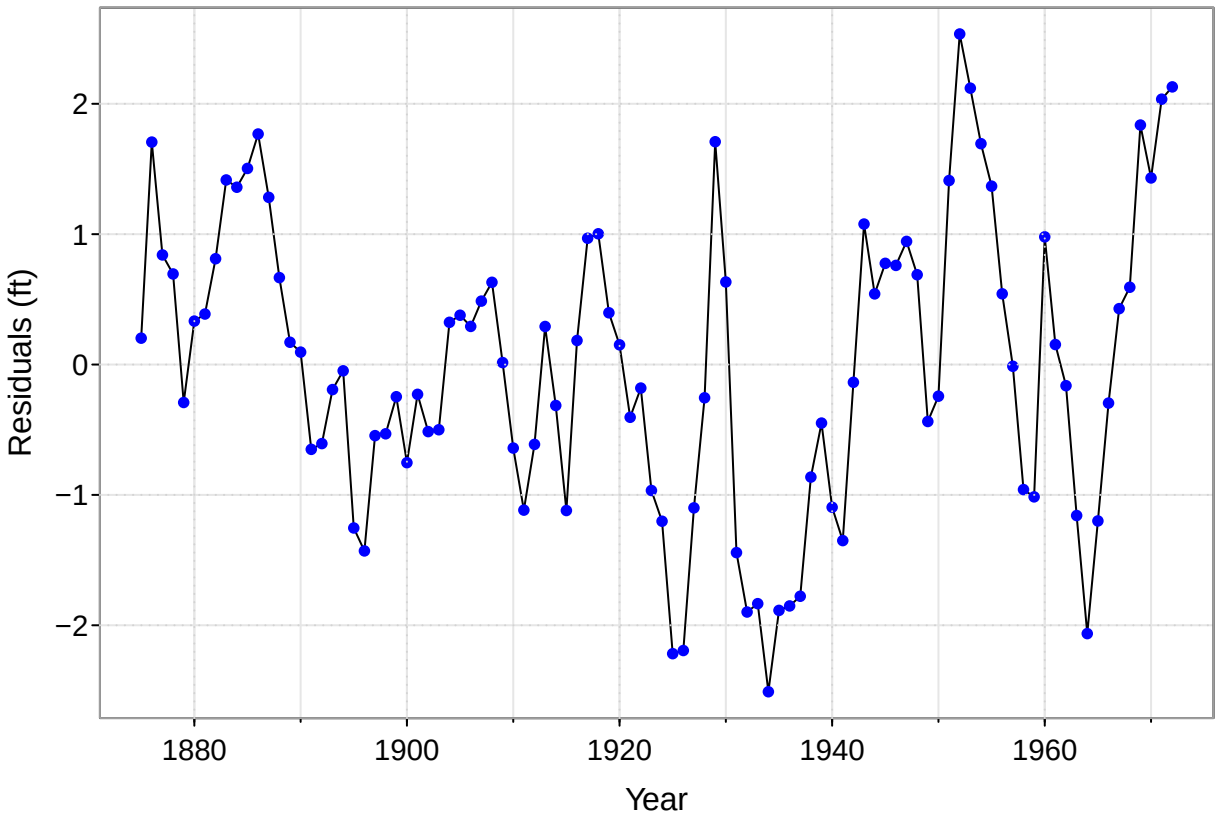
We can estimate the trend using simple linear regression and then examine the residuals

$$\hat{\eta}_t = y_t - \hat{y}_t.$$

```
library(astsa) # Load additional time-series functions
yr <- time(LakeHuron) # Extract the observation years
fit <- lm(LakeHuron ~ yr) # Fit a linear trend

plot(LakeHuron, ylab = "Level (ft)", xlab = "Year", las = 1) # Plot the original series
abline(fit, col = "blue", lwd = 2) # Add the estimated linear trend
```




```
res <- ts(residuals(fit), start = start(LakeHuron), frequency = 1) # Store residuals as a time series
tsplot(res, ylab = "Residuals (ft)", xlab = "Year", las = 1) # Plot detrended series
points(res, cex = 0.8, col = "blue", pch = 16) # Highlight annual residuals
grid()
```



Removing the trend does **not** make the remaining observations independent. The residual plot suggests that nearby values tend to move together: a positive residual is often followed by another positive residual, and similarly for negative residuals.

This is an example of **temporal dependence**.

Exploring Temporal Dependence with Lag Plots

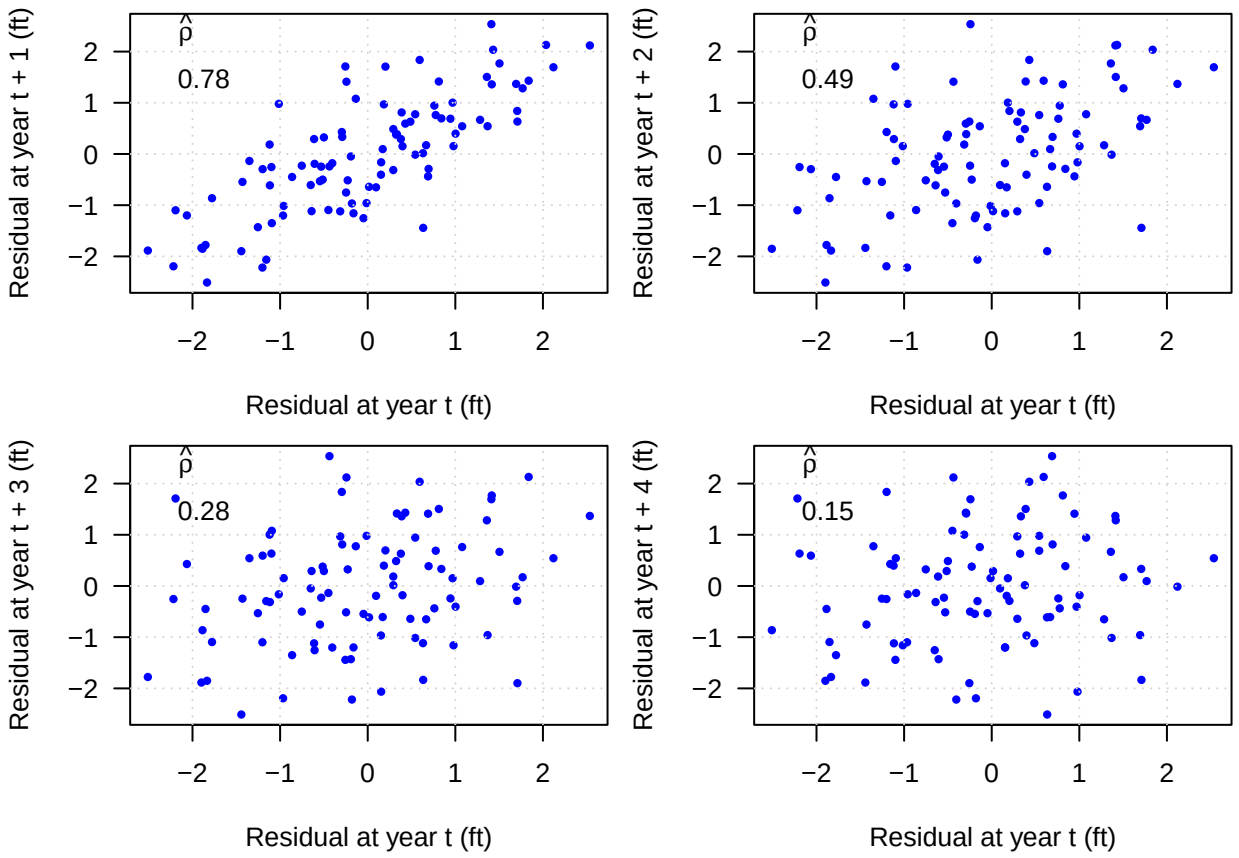
One simple way to examine temporal dependence is to compare observations separated by a fixed time lag.

For lag h, we compare

$$\hat{\eta}_t \quad \text{with} \quad \hat{\eta}_{t+h}.$$

If the two quantities are related, the scatterplot will show structure rather than a random cloud of points. We examine lags $h = 1, 2, 3,$ and 4 years.

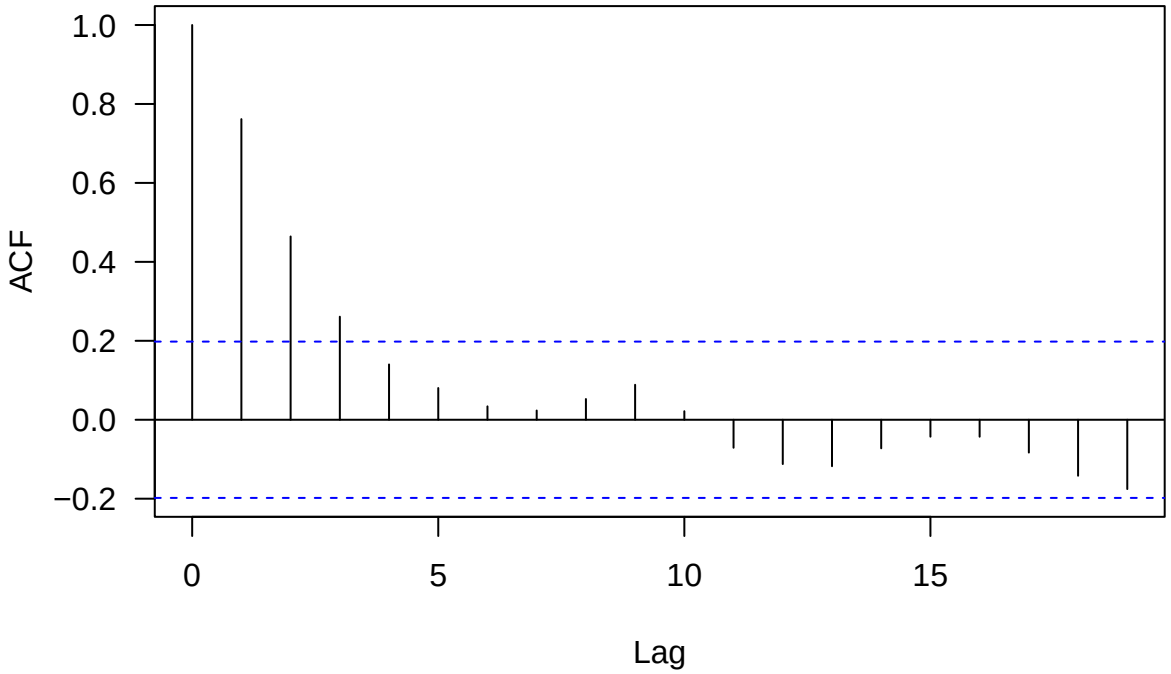
```
n <- length(res); h <- 1:4 # Series length and lags to examine
par(mfrow = c(2, 2), mar = c(4, 4, 0.8, 0.6)) # Arrange four lag plots

for (i in h){
  xlag <- res[1:(n-i)]; ylag <- res[(i+1):n] # Match residuals separated by lag i
  plot(xlag, ylag, pch = 16, col = "blue", cex = 0.7, las = 1,
       xlab = "Residual at year t (ft)", ylab = paste("Residual at year t +", i, "(ft)"))
  legend("topleft", legend = round(cor(xlag, ylag), 2),
         title = expression(hat(rho)), bty = "n") # Display sample lag correlation
  grid()
}
```



The lag plots indicate positive association, particularly at shorter lags. In other words, observations close together in time tend to be more similar than observations farther apart.

This dependence can be summarized systematically using the **autocorrelation function (ACF)**.

```
acf(res, las = 1, main = "") # Sample autocorrelation as a function of time lag
```



The ACF measures the correlation between observations separated by different time lags. For example, the lag-1 autocorrelation measures the association between values one year apart.

We will study autocovariance and autocorrelation in detail later in the course and use them to identify and model temporal dependence.

> **Key idea:** Unlike many introductory statistical methods, time series analysis generally cannot assume that observations are independent.

Time series analysis is largely concerned with understanding and modeling this **dependence among observations over time**.

Time Series Models

To move from exploratory analysis to statistical inference, we need a probabilistic model.

A **time series model** describes how an observed series $\{y_t\}$ could have been generated. We therefore view the observed data as one realization of a stochastic process

$$\{Y_t : t \in T\},$$

where each Y_t is a random variable indexed by time.

For observations at T time points, we can think of

$$(Y_1, Y_2, \ldots, Y_T)'$$

as a T-dimensional random vector. Unlike a conventional multivariate dataset, however, we typically observe only **one realization of this vector**.

This creates a fundamental challenge: **how can we learn the probabilistic structure of a time series from only one observed realization?**

Stationarity

Without additional assumptions, estimating the full probabilistic structure of $\{Y_t\}$ from a single realization is generally impossible. We therefore need assumptions that impose structure across time.

One of the most important is stationarity.

Broadly, stationarity means that the statistical behavior of the process is **stable over time**. Rather than depending on the specific calendar time t, dependence is determined primarily by the **time separation, or lag**, between observations.

Most observed time series are not stationary in their original form. Trend, seasonality, changing variance, and structural changes can all produce nonstationarity. A common strategy is therefore to model or remove these systematic components—for example, through **detrending** or **seasonal adjustment**—and then model the remaining process $\{\eta_t\}$ as approximately stationary.

A particularly important form is **second-order (weak) stationarity**. A process $\{Y_t\}$ is second-order stationary if

$$E(Y_t) = \mu$$

is constant over time and

$$\text{Cov}(Y_t, Y_{t+h}) = \gamma(h)$$

depends only on the lag h, not on the particular time t.

Thus, under second-order stationarity,

$$\boxed{\text{same lag} \quad \Longrightarrow \quad \text{same covariance structure.}}$$

This assumption allows us to learn about temporal dependence by combining information from pairs of observations having the same lag.

Objectives of Time Series Analysis

Once we have an appropriate statistical model for the temporal structure, we can use it for several purposes.

Modeling

The first objective is to find a statistical model that adequately describes the important features of the observed series, including its systematic components and temporal dependence.

For the Lake Huron example, this means modeling both the long-term trend and the dependence among lake levels across years.

A fitted model also enables statistical inference. For example:

> *Is there evidence of a long-term decrease in Lake Huron levels?*

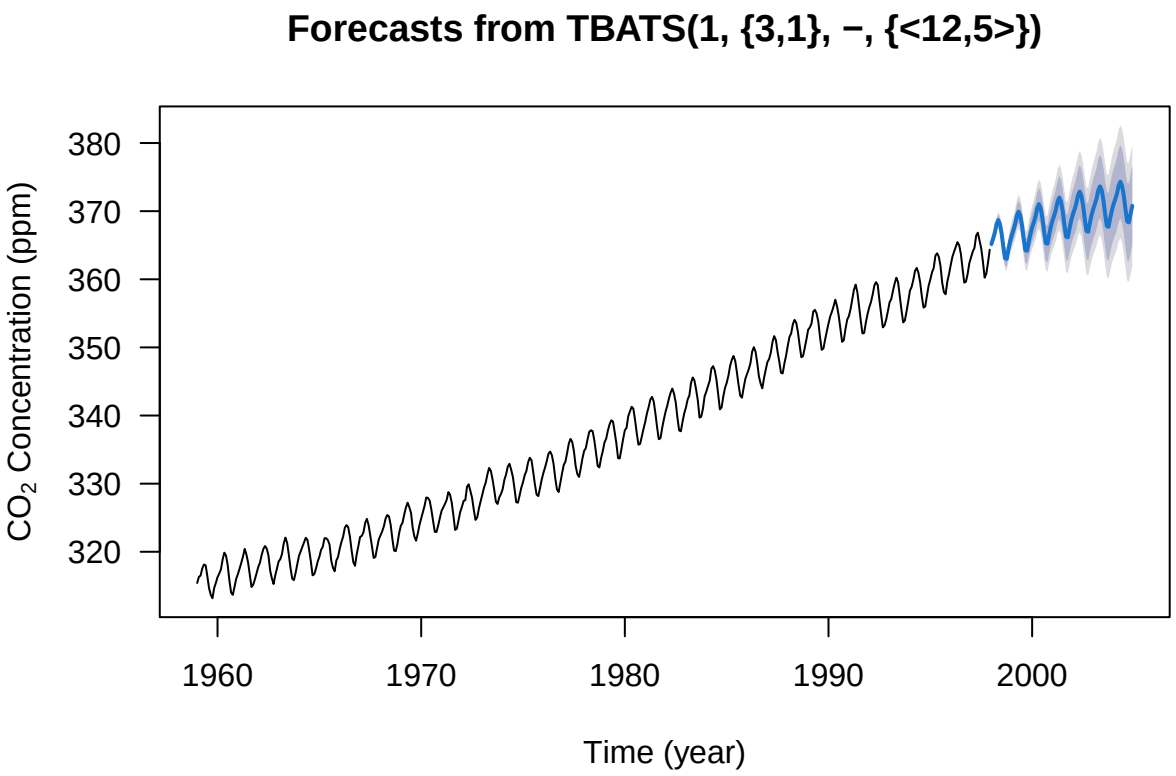
Forecasting

Perhaps the most familiar objective of time series analysis is **forecasting**. Given observations up to the present time, we want to predict future values.

A statistical time series model allows us to produce both **point forecasts** and measures of **forecast uncertainty**, such as prediction intervals.

For example, we can fit a model to the atmospheric CO_2 series and forecast future concentrations.

```
library(forecast) # Load forecasting functions
TBATSfit <- tbats(co2, use.box.cox = FALSE, use.trend = FALSE, use.damped.trend = FALSE,
                  seasonal.periods = 12) # Fit a seasonal TBATS model
fc <- forecast(TBATSfit, h = 84) # Forecast the next 84 months
plot(fc, las = 1, xlab = "Time (year)",
     ylab = expression(paste("CO"[2], " Concentration (ppm)"))) # Plot forecasts and uncertainty
```



A useful forecast should therefore answer not only

What is likely to happen?

but also

How uncertain are we about that prediction?

Adjustment

Time series models can be used to estimate and remove systematic components that may obscure other features of interest.

A common example is **seasonal adjustment**, where the seasonal component is estimated and removed so that the underlying trend and other movements can be examined more clearly.

Simulation

Once a time series model provides an adequate representation of a process, we can use it to generate **new realizations with similar statistical properties**.

Repeated simulation is useful for studying possible future behavior, evaluating statistical procedures, assessing uncertainty, and approximating the behavior of complex physical processes.

Control

In some applications, we observe a process over time and can adjust input or control variables to keep the process close to a desired target.

This is particularly important in **statistical process and quality control**, where time-series methods can help detect changes in a process and guide corrective action.

References

Breckling, Jens. 2012. *The Analysis of Directional Time Series: Applications to Wind Speed and Direction.* Vol. 61. Springer Science & Business Media.

Brockwell, Peter J, Peter J Brockwell, Richard A Davis, and Richard A Davis. 2002. *Introduction to Time Series and Forecasting.* Springer.

Huang, Whitney K, Yu-Min Chung, Yu-Bo Wang, Jeff E Mandel, and Hau-Tieng Wu. 2022. "Airflow Recovery from Thoracic and Abdominal Movements Using Synchrosqueezing Transform and Locally Stationary Gaussian Process Regression." *Computational Statistics & Data Analysis* 174: 107384.

Jia, Yisu, Stefanos Kechagias, James Livsey, Robert Lund, and Vladas Pipiras. 2021. "Latent Gaussian Count Time Series." *Journal of the American Statistical Association*, no. just-accepted: 1–28.